Ivaykina Darya Lab2

1. How does REST fundamentally differ from the socket protocol of Laboratory Work 1? What does it give that every request is self-contained?

In Laboratory Work 1 the client opened one long connection and stayed connected until it sent QUIT, and the server used its own text commands. In REST every request is a separate, complete message: it has a URL, a method, a JSON body, and the server answers with a code (200, 400, 404, 202) and then forgets the request.

Because each request contains everything the server needs, any copy of the service can answer any request, so it is easy to run several copies behind a load balancer. It is also easy to test with standard tools like curl or a browser.

2. Why does the job-submission endpoint return code 202 and not 200?

Code 200 means that the work is done and the result is in the response. When a job is submitted, it is only placed in the queue, and the worker will process it later. Code 202 Accepted means that the job was received but is not finished yet. The client gets the job id and asks for the result later with GET /jobs/{id}.

3. Why is a queue needed if the job could be done directly inside the request? When does it matter?

If the job is done inside the request, the client has to wait until it ends, a slow job can cause a timeout, and the server is busy and cannot serve other users. With a queue the API answers immediately and a separate worker does the slow work in the background. Jobs are not lost if the worker is busy or stopped, and more workers can be added.

It matters for slow tasks (reports, emails, image processing) and when the number of requests suddenly grows.

4. What happens if the worker crashes after receiving a message but before deleting it from the queue?

In my implementation the job file stays in queue/ until the result is saved, and only then it is deleted. If the worker crashes, the file is still in the folder, and after a restart the worker processes the job again, so the job is not lost.

The price is that a job can be processed twice, so repeating the work must be safe. In my worker a repeat just writes the same result file again. In Amazon SQS the same idea works with the visibility timeout: if a message is not deleted in time, it becomes visible in the queue again.